CS24303 • COMPLETE MASTER GUIDE

Data Mining Python Laboratory Guide

Hands-On Implementation of Apriori, Decision Trees & DBSCAN

HANDS-ON PRACTICAL LAB MASTER APPENDIX

Data Mining Laboratory & Python Implementation Master Guide

Complete Python Algorithms for Apriori Association Rules, Decision Tree Classification & DBSCAN Clustering

Lab Experiment 1: Production-Grade Apriori Algorithm Implementation in Python

```
import pandas as pd
from itertools import combinations

def get_frequent_1_itemsets(transactions, min_sup_count):
    item_counts = {}
    for transaction in transactions:
        for item in transaction:
            item_counts[frozenset([item])] = item_counts.get(frozenset([item]), 0) + 1
    return {item: count for item, count in item_counts.items() if count ⩾ min_sup_count}

def apriori_gen(Lk_minus_1, k):
    candidates = set()
    lk_list = list(Lk_minus_1.keys())
    for i in range(len(lk_list)):
        for j in range(i + 1, len(lk_list)):
            union_set = lk_list[i] | lk_list[j]
            if len(union_set) == k:
                subsets = [frozenset(s) for s in combinations(union_set, k - 1)]
                if all(s in Lk_minus_1 for s in subsets):
                    candidates.add(union_set)
    return candidates

def run_apriori(dataset, min_support=0.4):
    num_trans = len(dataset)
    min_sup_count = min_support * num_trans
    L1 = get_frequent_1_itemsets(dataset, min_sup_count)

    all_frequent = dict(L1)
    current_L = L1
    k = 2

    while current_L:
        candidates = apriori_gen(current_L, k)
        candidate_counts = {}
        for trans in dataset:
            trans_set = set(trans)
            for cand in candidates:
                if cand.issubset(trans_set):
                    candidate_counts[cand] = candidate_counts.get(cand, 0) + 1

        current_L = {cand: count for cand, count in candidate_counts.items() if count ⩾ min_sup_count}
        all_frequent.update(current_L)
        k += 1

    return all_frequent

# Example Run
dataset = [
    ['Milk', 'Onion', 'Nutmeg', 'Kidney_Beans', 'Eggs', 'Yogurt'],
    ['Dill', 'Onion', 'Nutmeg', 'Kidney_Beans', 'Eggs', 'Yogurt'],
    ['Milk', 'Apple', 'Kidney_Beans', 'Eggs'],
    ['Milk', 'Unicorn', 'Corn', 'Kidney_Beans', 'Yogurt'],
    ['Corn', 'Onion', 'Onion', 'Kidney_Beans', 'Ice_cream', 'Eggs']
]

frequent_patterns = run_apriori(dataset, min_support=0.6)
for itemset, count in frequent_patterns.items():
    print(f"Frequent Itemset: {set(itemset)} (Support = {count/len(dataset):.2f})")
```

Lab Experiment 2: End-to-End Decision Tree Induction & Evaluation

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# Dataset generation
np.random.seed(42)
X = np.random.randn(200, 4)
y = (X[:, 0] + X[:, 1] > 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

clf = DecisionTreeClassifier(criterion='entropy', max_depth=4)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Lab Experiment 3: Density-Based Spatial Clustering (DBSCAN) in Python

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

# Non-convex crescent moon dataset
X_moons, _ = make_moons(n_samples=300, noise=0.08, random_state=42)

dbscan = DBSCAN(eps=0.2, min_samples=5)
clusters = dbscan.fit_predict(X_moons)

n_clusters = len(set(clusters)) - (1 if -1 in clusters else 0)
n_noise = list(clusters).count(-1)
print(f"Discovered Clusters: {n_clusters}, Detected Noise Outliers: {n_noise}")
```